Database – No SQL – Mongodb

Session 7 –

1. **Create a MongoDB collection called 'restaurants' and insert at least 10 documents with fields: name, cuisine, location, and rating. Run a find query to retrieve all restaurants with cuisine 'Italian'.**

Ans:

```
db.createCollection("restaurants")

db.restaurants.insertMany([
    {
        name: "Pasta Paradise",
        cuisine: "Italian",
        location: "Ahmedabad",
        rating: 4.6
    },
    {
        name: "Pizza Corner",
        cuisine: "Italian",
        location: "Surat",
        rating: 4.5
    },
    {
        name: "Bella Italia",
        cuisine: "Italian",
        location: "Vadodara",
        rating: 4.8
    },
    {
        name: "Dragon Palace",
        cuisine: "Chinese",
        location: "Ahmedabad",
        rating: 4.3
    },
    {
        name: "Great Wall",
        cuisine: "Chinese",
        location: "Surat",
        rating: 4.4
    },
    {
        name: "Spice Hub",
        cuisine: "Indian",
        location: "Ahmedabad",
        rating: 4.2
```

```
    },
    {
        name: "Curry House",
        cuisine: "Indian",
        location: "Surat",
        rating: 4.5
    },
    {
        name: "Burger Point",
        cuisine: "American",
        location: "Vadodara",
        rating: 4.1
    },
    {
        name: "Pizza Palace",
        cuisine: "Pizza",
        location: "Surat",
        rating: 4.9
    },
    {
        name: "Cheese Burst",
        cuisine: "Pizza",
        location: "Surat",
        rating: 4.7
    }
])
```

Find all Italian restaurants:

```
db.restaurants.find({
    cuisine: "Italian"
}).pretty()
```

2. **Add a single field index on the 'cuisine' field in the 'restaurants' collection using the createIndex() method. Measure and compare the query execution time for finding all 'Italian' restaurants before and after adding the index.

Hint: Use db.restaurants.find({cuisine: 'Italian'}) and db.restaurants.createIndex({cuisine: 1}). Use the .explain('executionStats') method to check execution time.**

Ans:

Before creating the index, run:

```
db.restaurants.find({
    cuisine: "Italian"
}).explain("executionStats")
```

Now create the index:

```
db.restaurants.createIndex({
    cuisine: 1
})
```

Run the same query again:

```
db.restaurants.find({
    cuisine: "Italian"
}).explain("executionStats")
```

3. **Create a compound index on the 'cuisine' and 'location' fields in the 'restaurants' collection. Write a query to find all 'Chinese' restaurants in 'Ahmedabad' and check if the compound index is being used by the query planner.**

Ans:

Create the compound index:

```
db.restaurants.createIndex({
    cuisine: 1,
    location: 1
})
```

Query Chinese restaurants in Ahmedabad:

```
db.restaurants.find({
    cuisine: "Chinese",
    location: "Ahmedabad"
}).explain("executionStats")
```

Look through the output for:

IXSCAN

and an index name similar to:

cuisine_1_location_1

You can also explicitly inspect the query plan:

```
db.restaurants.find({
    cuisine: "Chinese",
    location: "Ahmedabad"
}).explain("queryPlanner")
```

4. **Drop the index you created on the 'cuisine' field and observe how the query performance changes when searching for a specific cuisine. Record your observations in 2-3 lines.**

Ans:

First list the indexes:

db.restaurants.getIndexes()

Then remove the single-field index:

db.restaurants.dropIndex("cuisine_1")

Check the indexes again:

db.restaurants.getIndexes()

Now test the Italian query:

db.restaurants.find({

  cuisine: "Italian"

}).explain("executionStats")

5. **Use ChatGPT to generate a MongoDB query that would efficiently fetch the top 5 highest rated 'Pizza' restaurants in 'Surat', making sure to use any relevant indexes. Paste the generated query and explain in one line how indexing helps in this scenario.**

Ans:

```
db.restaurants.find(
    {
        cuisine: "Pizza",
        location: "Surat"
    },
    {
        _id: 0,
        name: 1,
        cuisine: 1,
        location: 1,
        rating: 1
    }
).sort({
    rating: -1
}).limit(5)
```

For this query, create an index that supports the filtering and sorting:

```
db.restaurants.createIndex({
    cuisine: 1,
    location: 1,
    rating: -1
})
```

Then verify that MongoDB can use it:

```
db.restaurants.find(
    {
        cuisine: "Pizza",
        location: "Surat"
    }
).sort({
    rating: -1
}).limit(5).explain("executionStats")
```